

CUADERNO DE DIVULGACIÓN · DEEP REINFORCEMENT
LEARNING

Arkanoid DRL Learning Lab

Cuatro algoritmos de aprendizaje por refuerzo aprendiendo a jugar al Arkanoid en el navegador, con redes neuronales **reales** sobre TensorFlow.js y aceleración **WebGPU**. Sin trucos: el agente no conoce las reglas del juego; las descubre jugando.

DQN

PPO

SAC

World Model

TensorFlow.js

WebGPU

AUTOR

Roberto Canales Mora

LICENCIA

MIT

REPOSITORIO

github.com/robertocanalesmora/arkanoid-drl-learning-lab

INTRODUCCIÓN

De un ladrillo a una política

Qué es esto, quiénes son los protagonistas y cómo está pensado para aprenderse.

El **aprendizaje por refuerzo** (RL, *Reinforcement Learning*) es la rama de la IA en la que un **agente** aprende a base de **premios y castigos**: prueba acciones, observa la recompensa y ajusta su comportamiento para acumular el máximo premio a largo plazo. Aquí lo vemos sobre un Arkanoid simplificado, donde el agente solo puede mover una pala a izquierda/derecha o dejarla quieta, y debe aprender a devolver la pelota y romper ladrillos **sin que nadie le explique las reglas**.

Los protagonistas, en una frase

Entorno	El juego de Arkanoid: recibe una acción, avanza la física un paso y devuelve una recompensa.
Estado	Lo que el agente "ve" en cada instante: 6 números (posición y velocidad de la pelota, posición de la pala...).
Acción	Una de 3: mover la pala a la izquierda, mantener, o a la derecha.
Recompensa	El número que premia (romper un ladrillo: +1) o castiga (perder la pelota: -1) cada paso.
Agente	La red (o redes) neuronal que decide qué acción tomar y que aprende de la experiencia.
Política	La estrategia del agente: la función que, dado un estado, dice qué acción conviene.

El hilo conductor: construir → entrenar → medir → mejorar

Cada algoritmo de este cuaderno se ha tratado igual: se **construye**, se **entrena de verdad**, se **leen sus métricas** (recompensa, pérdida, exploración, throughput) y, si no aprende, se **ajusta** (hiperparámetros, arquitectura, regla de actualización) hasta que mejora de forma medible. Esa misma disciplina — proponer, entrenar, medir, quedarse con lo que funciona — es la que aplicamos con agentes de IA para afinar el sistema.

Todas las cifras de este cuaderno son reales, medidas ejecutando el propio laboratorio (en el navegador con WebGPU y en Node con CPU). Las capturas son de la aplicación funcionando; las curvas, de entrenamientos reales. Nada está inventado ni maquillado.

Lo que vas a poder seguir



Cómo arranca y se ejecuta el juego

La simulación de paso fijo, el estado que ve el agente y cómo se reinicia un episodio.



Cómo enganchamos los controles

El agente sustituye al jugador: su red decide las 3 acciones para cientos de partidas a la vez.



Cómo se guardan los datos

La transición $(s, a, r, s', done)$, el replay buffer y el rollout: la memoria del aprendizaje.



Cómo se entrena la red y se actualizan las gráficas

El paso de gradiente, la red objetivo y el flujo desde el entrenamiento hasta las curvas en pantalla.

Del juego a la red neuronal

El ciclo de vida completo de una iteración de entrenamiento, paso a paso.

Antes de ver los algoritmos uno a uno, conviene entender la **maquinaria común** que todos comparten. Si entiendes este ciclo, sabrás trasladar el mismo esqueleto a cualquier otro proyecto de RL: cambia el juego y el algoritmo, pero las piezas (entorno, agente, memoria, bucle, métricas) son siempre las mismas.

1 · Cómo arranca y se ejecuta el juego

El entorno es una **simulación pura**: no dibuja nada, solo calcula física. Vive en `entornoArkanoid.js`. Al crearse o reiniciarse coloca los 28 ladrillos (4 filas x 7 columnas), lanza la pelota con una dirección aleatoria y sitúa la pala. La clave de diseño es que la física es de **paso fijo**: cada llamada a `paso(accion)` avanza **exactamente un instante** de simulación. La "velocidad" que verás en la interfaz no acelera la física (eso cambiaría la dinámica y rompería el aprendizaje): solo decide cuántos pasos se ejecutan por fotograma.

`src/entorno/entornoArkanoid.js`

EL MOTOR DEL JUEGO

```
// Avanza un paso de simulación aplicando la acción del agente.
paso(accion) {
  if (this.estaTerminado()) return { recompensa: 0, done: true };
  this.pasos++;
  this.recompensaPaso = RECOMPENSAS.PASO;           // 0 por paso (no penaliza sobrevivir)
  this._aplicarAccion(accion);                       // mueve la pala ← · →
  this._moverPelota();                               // x += vx ; y += vy (paso fijo)
  this._colisionParedes();
  this._colisionPala();                             // +0.10 si devuelve la pelota
  this._colisionLadrillos();                         // +1.0 por ladrillo roto
  this._verificarFin();                              // -1.0 si pierde la pelota; +5.0 si limpia
  el nivel
  this._aplicarShaping();                            // recompensa densa: acercarse a la pelota
  this.recompensaEpisodio += this.recompensaPaso;
  return { recompensa: this.recompensaPaso, done: this.estaTerminado() };
}
```

Una sola función concentra toda la regla del juego. Devuelve la **recompensa** de ese paso y si el episodio ha **terminado** (done).

🎮 PARTIDA OBSERVADA · ENV 0

● jugando



Recompensa episodio: **0.01** Ladrillos: 0/28 Pasos: **1**

La partida observada: pelota, pala (degradado azul-violeta) y los 28 ladrillos. Abajo, la recompensa del episodio, los ladrillos rotos y los pasos.

El estado: lo único que ve el agente

El agente no ve píxeles, ve un vector de **6 números normalizados** a $\sim[-1, 1]$: posición x e y de la pelota, su velocidad (vx, vy), la posición de la pala y la diferencia pelota-pala. Normalizar (que todo viva en el mismo rango) es lo que permite que una red pequeña aprenda rápido.

src/entorno/entornoArkanoid.js · obtenerVectorEstado()

EL ESTADO

```
return [
  p.x * 2 - 1, p.y * 2 - 1,           // posición de la pelota → [-1,1]
  p.vx / CFG.VELOCIDAD_PELOTA, p.vy / CFG.VELOCIDAD_PELOTA, // velocidad → [-1,1]
  this.pala.x * 2 - 1,               // posición de la pala
  limitar((p.x - this.pala.x) * 2, -1, 1), // distancia horizontal pelota↔pala
];
```

6 features. Pocas, pero suficientes para describir la situación. El reto del agente es mapear estos 6 números a la mejor de 3 acciones.

2 · Cómo enganchamos los controles: el agente "juega"

En un Arkanoid normal, un humano pulsa teclas. Aquí el **agente sustituye al jugador**: en cada paso, su red recibe el estado y devuelve la acción. Y como queremos aprender deprisa, no jugamos una partida, sino **cientos a la vez**: el estado de los N entornos se apila en un único tensor $[N \times 6]$ y la red predice las N acciones **de una sola pasada** por la GPU (inferencia en lote). Esto es lo que hace que WebGPU rente: una llamada grande en vez de N pequeñas.

src/agentes/agenteDQN.js · seleccionarAcciones()

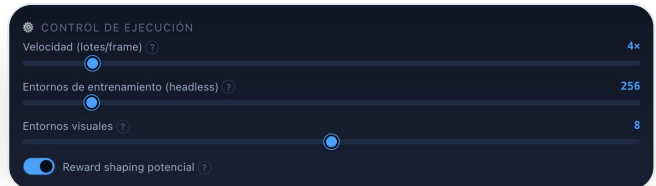
EL MANDO DEL AGENTE

```
seleccionarAcciones(estadosFlat, n, { entrenar = true } = {}) {
  const eps = entrenar ? this.epsilon : 0;           // ε = probabilidad de explorar al azar
  const greedy = tf.tidy(() => {
    const sT = this._tensorEstados(estadosFlat, n);   // tensor [N × 6]
    return this.redPolitica.predict(sT).argMax(1).dataSync(); // mejor acción de cada entorno
  });
  const acciones = new Int32Array(n);
  for (let i = 0; i < n; i++)                         // ε-greedy: a veces exploro, casi
    acciones[i] = Math.random() < eps ? (Math.random()*this.numAcciones)|0 : greedy[i]; // siempre exploto
  return acciones;                                     // N acciones, una por partida
}
```

El agente convierte estados en acciones. **ε-greedy**: con probabilidad ε actúa al azar (explorar), si no elige la mejor acción conocida (explotar).



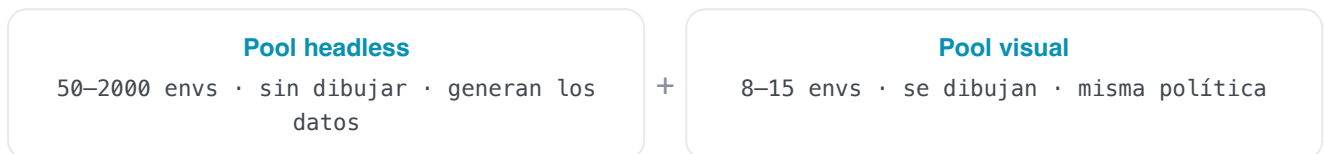
Eliges qué cerebro pilota: DQN, PPO, SAC o World Model.
Cambiarlo recrea el agente desde cero.



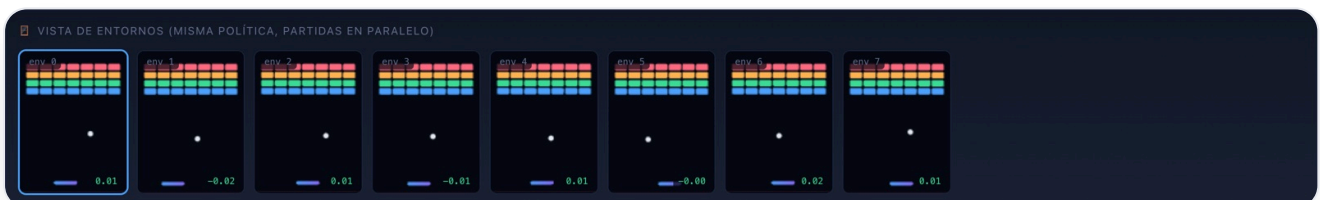
Los mandos: velocidad (lotes/frame), n° de entornos de entrenamiento y de observación, y el reward shaping.

3 · Dos mundos: entrenar rápido y observar bonito

Si atáramos el aprendizaje a lo que se dibuja, estaríamos limitados a 8 ó 15 partidas. Por eso hay **dos pools de entornos** separados (`gestorEntornos.js`):



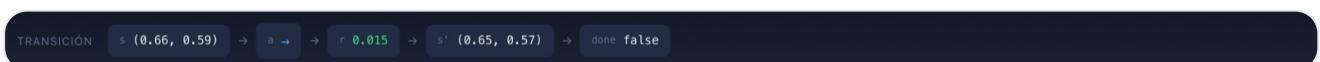
El pool **headless** (sin canvas) corre cientos de partidas para alimentar a la red. El pool **visual** ejecuta la **misma política** solo para que tú veas cómo juega. Es una ventana de observación, no el motor.



La rejilla de entornos visuales: muchas partidas en paralelo con la misma política. Puedes pinchar una para verla grande arriba.

4 · Cómo se guardan los datos para el entrenamiento

La unidad de experiencia es la **transición**: $(s, a, r, s', done)$ — "estaba en el estado s , hice la acción a , gané r , acabé en s' , y el episodio terminó (o no)". Cada paso de cada entorno produce una. El gestor las empaqueta en **arrays planos** (muy eficientes para crear tensores) y se las pasa al agente para que las guarde.



La tira de transición muestra en vivo el $(s \rightarrow a \rightarrow r \rightarrow s' \rightarrow done)$ del entorno observado: el ladrillo de información con el que se aprende.

¿Dónde se guardan? Depende de la familia del algoritmo:

Off-policy → Replay buffer

DQN, SAC y World Model guardan **100.000** transiciones en una memoria circular y entrenan muestreando lotes aleatorios. Reutilizan cada experiencia muchas veces y rompen la correlación temporal.

On-policy → Rollout buffer

PPO acumula un **rollout** (256 pasos $\times$ N entornos), calcula las ventajas con GAE, entrena unas pocas épocas y **descarta** los datos: solo valen mientras la política no cambie.

```
// Inserta un lote de transiciones en formato plano (buffer circular).
agregarLote(lote) {
  const n = lote.recompensas.length;
  for (let i = 0; i < n; i++) {
    const baseDst = this.pos * this.dim, baseSrc = i * this.dim;
    for (let k = 0; k < this.dim; k++) {
      this.s[baseDst + k] = lote.estados[baseSrc + k]; // estado s
      this.s2[baseDst + k] = lote.siguietes[baseSrc + k]; // estado siguiente s'
    }
    this.a[this.pos] = lote.acciones[i]; // acción a
    this.r[this.pos] = lote.recompensas[i]; // recompensa r
    this.done[this.pos] = lote.terminados[i]; // done
    this.pos = (this.pos + 1) % this.capacidad; // circular: pisa lo más viejo
    if (this.tam < this.capacidad) this.tam++;
  }
}
```

La experiencia se guarda en columnas (s, a, r, s', done) para poder muestrear lotes y convertirlos en tensores sin copias caras.

5 · El bucle de entrenamiento (la pieza que lo une todo)

El orquestador.js no sabe de algoritmos ni de pantallas: solo repite un ciclo. Esta es **una iteración** (un "lote"): se observan los estados, el agente decide, el juego avanza, se guarda la experiencia y el agente entrena. Es el corazón que late decenas de veces por segundo.

```
async ejecutarLote() {
  const n = gestor.numHeadless;
  const estados = gestor.obtenerEstadosEntrenamiento(); // 1· observar los N estados
  const acciones = this.agente.seleccionarAcciones(estados, n); // 2· la red decide N acciones
  const resultado = gestor.aplicarAcciones(acciones); // 3· el juego avanza N pasos
  this.agente.almacenarExperiencia(resultado); // 4· guardar (s,a,r,s',done)
  this.mtricas.registrarEpisodios(resultado.episodios); // 5· anotar episodios
  terminados
  const metrica = await this.agente.entrenar(); // 6· UN paso de aprendizaje
  this.pasoGlobal += n;
  // 7· cada 250 pasos: registrar una traza y avisar a la UID por el bus de eventos
}
```

Siempre los mismos 7 gestos. Cambiar de algoritmo solo cambia qué hace entrenar() por dentro; el bucle es idéntico.

6 · Cómo se entrena la red neuronal (el paso de gradiente)

Entrenar una red es ajustar sus **pesos** para que se equivoque menos. En RL no tenemos "respuestas correctas": construimos un **objetivo** a partir de la recompensa y de lo que la propia red cree que vale el futuro (ecuación de Bellman), y empujamos la predicción hacia ese objetivo. El truco para que no diverja es usar una **red objetivo**: una copia que se mueve despacio y nos da un blanco estable al que apuntar.

```
objetivo = r + γ · (1 - done) · Q-(s', a*) // γ=0.99; Q- = red objetivo; a* = mejor acción
```



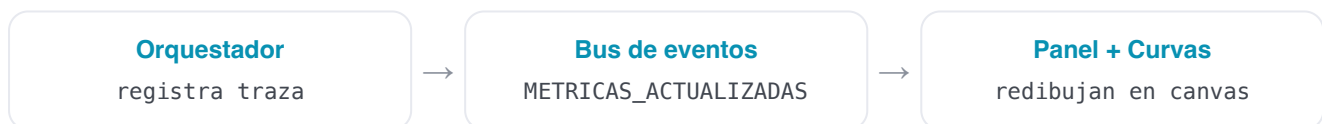
```
// El objetivo TD se calcula SIN gradiente (es el "blanco" estable).
const objetivo = tf.tidy(() => {
  const aStar = this.redPolitica.predict(s2T).argMax(1); // Double DQN: la red online elige a*
  const qSig = this.redObjetivo.predict(s2T)...sum(1); // la red objetivo la evalúa
  return rT.add(qSig.mul(gamma).mul(noTerm)); //  $r + \gamma \cdot (1 - \text{done}) \cdot Q'(s', a^*)$ 
});
// Un paso de descenso de gradiente: acerca Q(s,a) al objetivo (pérdida Huber).
const lossT = this.optimizador.minimize(() => {
  const qa = this.redPolitica.predict(sT).mul(aOneHot).sum(1);
  return tf.losses.huberLoss(objetivo, qa);
}, true, this._varsPolitica);
// La red objetivo sigue a la online MUY despacio (Polyak): estabilidad.
actualizacionSuave(this.redPolitica, this.redObjetivo, tau); //  $\tau = 0.01$ 
```

Bellman + red objetivo + pérdida Huber + actualización suave. Cada algoritmo cambia esta receta, pero la idea —empujar la predicción hacia un objetivo— se mantiene.

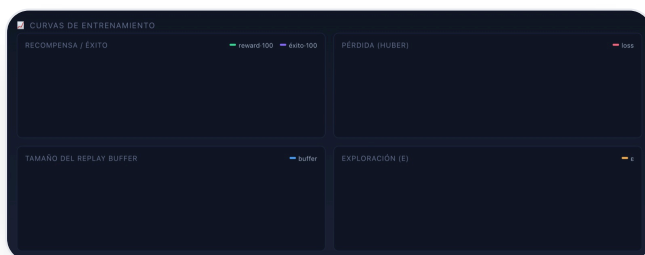
Higiene de memoria (TensorFlow.js). Cada operación crea tensores en la GPU. Si no se liberan, la memoria crece sin parar. Todo el laboratorio usa `tf.tidy` y un gestor de tensores; el resultado es un recuento **plano** de tensores durante todo el entrenamiento (verificado: 26 en DQN, 76 en SAC...).

7 · Cómo se van actualizando las gráficas

El orquestador no toca la pantalla: emite **trazas** por un **bus de eventos**. La UI está suscrita y, cuando llega una traza nueva (cada 250 pasos), refresca las tarjetas de métricas y **redibuja las cuatro curvas** en sus `<canvas>`. Así el motor y la interfaz quedan desacoplados: podrías entrenar sin pantalla (lo hacemos en Node) o cambiar la UI sin tocar el algoritmo.



Así se ve la diferencia entre el arranque (sin datos) y tras unos segundos de entrenamiento real:



Antes: recién arrancado, las curvas están vacías (aún no hay episodios ni gradientes).



Después: la recompensa sube (verde), la pérdida se asienta (rojo), el buffer se llena (azul) y ϵ decae (ámbar).

FLUJO DE DATOS



Off-policy: las experiencias se guardan en un replay buffer y se reutilizan en minibatches aleatorios, rompiendo la correlación temporal y mejorando la eficiencia de datos.

El propio laboratorio dibuja su flujo de datos, y se adapta al algoritmo (aquí, off-policy con replay buffer).

En una frase

Observar estados → la red decide acciones → el juego avanza y produce recompensas → se guardan transiciones → un paso de gradiente acerca la predicción a un objetivo de Bellman → se emite una traza que refresca las gráficas. Repite miles de veces por minuto. **Eso es entrenar un agente.**

DQN — Deep Q-Network

Aprende cuánto vale cada acción y elige la de mayor valor.

¿Qué es DQN?

DQN aprende una función $Q(s,a)$: dado un estado, estima la **recompensa futura total** que cabe esperar si tomamos cada acción y luego seguimos jugando bien. Con esa tabla de valores, actuar es trivial: elige la acción de mayor Q. Lo "deep" es que esa función Q no es una tabla, sino una **red neuronal**, capaz de generalizar a estados que nunca vio exactamente.



Una analogía

Como aprender a jugar a los dardos: cada tirada te da una puntuación (recompensa) y, con la práctica, intuyes cuánto "vale" apuntar a cada zona. DQN hace eso con números: estima el valor de cada movimiento y se queda con el mejor.

¿Para qué sirve? (en el mundo real)



Videojuegos de Atari

DQN fue el primer algoritmo que aprendió a jugar decenas de juegos de Atari a nivel humano solo a partir de píxeles.



Control discreto

Decisiones del tipo "encender/apagar/mantener": climatización, gestión de colas, semáforos.



Recomendación

Elegir el siguiente contenido a mostrar para maximizar el interés a largo plazo.



Energía

Decidir cuándo cargar/descargar una batería según precio y demanda.

Cómo funciona (su estructura)

La red es un **perceptrón multicapa** (MLP): toma los 6 números del estado, pasa por dos capas ocultas de 128 neuronas con activación ReLU, y produce **3 salidas**, una por acción (el valor Q de cada una).



$$Q(s,a) \leftarrow r + \gamma \cdot \max_{a'} Q(s', a') \quad // \text{ecuación de Bellman}$$

La idea clave: Bellman + estabilizadores

El valor de ahora se define en términos del valor del futuro. Para que ese "perseguirse a sí mismo" no diverja añadimos tres mejoras clásicas: **Double DQN** (separa quién elige y quién evalúa la

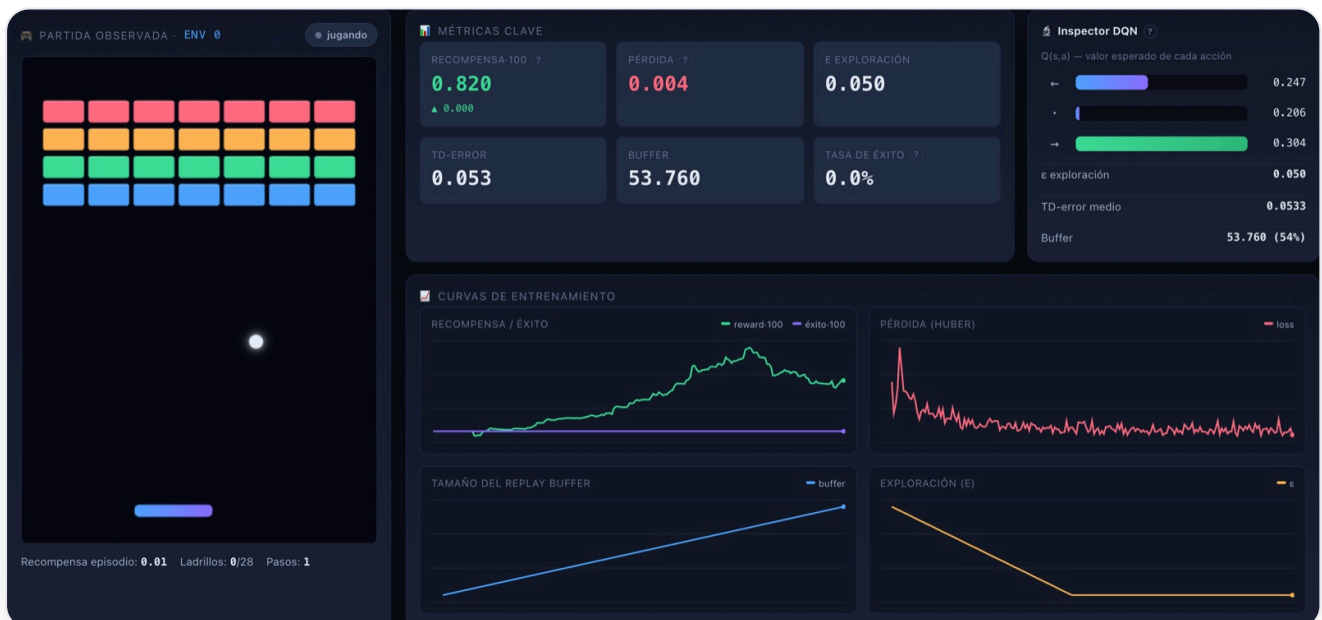
acción), **pérdida Huber** (menos sensible a errores grandes) y **red objetivo** con actualización suave.

El experimento: objetivo, entorno y parámetros

El entorno. Arkanoid: el agente observa 6 números, dispone de 3 acciones y recibe +1 por ladrillo, +0.1 por devolver la pelota y -1 por perderla. **El objetivo:** maximizar la recompensa acumulada por episodio.

AJUSTE	VALOR	QUÉ ES Y QUÉ EFECTO TIENE
learning_rate	8e-4	El tamaño de cada paso de aprendizaje. Grande = rápido pero inestable; pequeño = lento pero estable.
gamma (γ)	0.99	Cuánto importan las recompensas futuras frente a las inmediatas (0 = cortoplacista, $\rightarrow 1$ = previsor).
epsilon (ϵ)	1.0 $\rightarrow$ 0.05	Probabilidad de explorar al azar; decae a lo largo de 25.000 pasos hacia 0.05 (casi siempre explotar).
replay buffer	100.000	Cuántas experiencias recordamos para muestrear lotes aleatorios.
batch_size	128	Cuántas transiciones se usan en cada paso de gradiente.
tau (τ)	0.01	Velocidad a la que la red objetivo sigue a la principal (1% por paso): estabilidad.

La interfaz

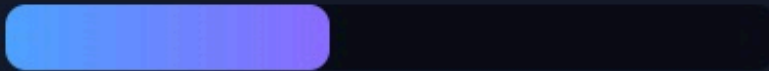


DQN entrenando: a la derecha, el inspector muestra los 3 valores Q (la acción greedy, en verde). Las curvas confirman que aprende: recompensa al alza, ϵ decayendo.

Inspector DQN ?

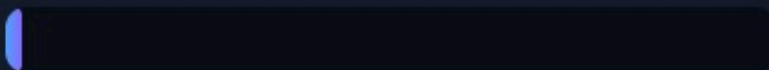
$Q(s,a)$ — valor esperado de cada acción

←



0.247

·



0.206

→



0.304

ϵ exploración

0.050

TD-error medio

0.0533

Buffer

53.760 (54%)

Inspector de DQN: $Q(s,a)$ de cada acción, el ϵ actual, el TD-error medio y el llenado del buffer.

Resultados y la mejora con agentes

En una ejecución real en el navegador (WebGPU), la recompensa media de los últimos 100 episodios subió de **~0.0 a ~0.94** en cuestión de segundos, con un throughput de **~24.000 experiencias/s**. El agente aprende con claridad a **seguir y devolver la pelota**; limpiar los 28 ladrillos (tasa de éxito > 0) exige entrenamientos largos.



Curvas reales de DQN: recompensa-100 (verde) al alza, pérdida Huber (rojo) que se asienta, buffer (azul) llenándose y ϵ (ámbar) decayendo.

⚠ El problema

Un DQN "de libro" **sobreestima** los valores Q (siempre coge el máximo, que arrastra el ruido) y la recompensa oscila o se desploma tras un buen pico.

✓ La mejora encontrada

Double DQN (la red online elige la acción y la objetivo la evalúa) + **Huber** + **soft update**. Resultado: aprendizaje más estable y sin fugas de memoria (26 tensores constantes).

Conclusiones

En una frase

DQN es **off-policy** y **basado en valor**: aprende $Q(s,a)$ reutilizando un **replay buffer**, explora con **ϵ -greedy** y se estabiliza con una **red objetivo**. Es el cimiento del RL profundo y el mejor punto de partida.

PPO — Proximal Policy Optimization

Optimiza la política directamente, pero con pasos prudentes.

¿Qué es PPO?

En lugar de estimar el valor de cada acción, PPO ajusta **directamente la política $\pi(\text{als})$** : una red que da la **probabilidad** de cada acción. Una segunda red, la **crítica**, estima el valor $V(s)$ del estado y sirve para juzgar si una acción fue mejor o peor de lo esperado (la **ventaja**). PPO es hoy el caballo de batalla del RL: estable y robusto.



Una analogía

Como mejorar tu técnica de escalada poco a poco: si cambias tu estilo de golpe, te caes. PPO da **pasos pequeños y controlados** sobre la política para no estropear lo ya aprendido.

¿Para qué sirve? (en el mundo real)



Robótica y control

Acciones continuas o discretas en robots, brazos y locomoción; es el algoritmo por defecto en muchas suites.



RLHF en LLMs

PPO se usó para alinear modelos de lenguaje con preferencias humanas (la "P" de ChatGPT entrenado con RLHF).



Juegos complejos

Dota 2 (OpenAI Five) y muchos entornos de simulación se entrenaron con PPO a gran escala.

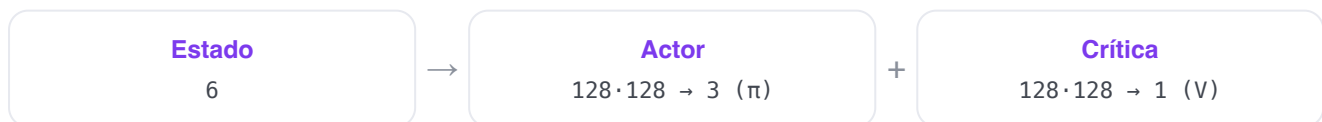


Optimización de procesos

Control de sistemas donde la estabilidad del entrenamiento es crítica.

Cómo funciona (su estructura)

Dos redes MLP independientes: el **actor** ($6 \rightarrow 128 \rightarrow 128 \rightarrow 3 \text{ logits} \rightarrow \text{softmax} = \text{probabilidades}$) y la **crítica** ($6 \rightarrow 128 \rightarrow 128 \rightarrow 1 = \text{valor del estado}$).



$$L = E[\min(\rho \cdot A, \text{clip}(\rho, 1-\epsilon, 1+\epsilon) \cdot A)] \quad // \quad \rho = \pi_{\text{nueva}} / \pi_{\text{vieja}} ; A = \text{ventaja}$$

La idea clave: el recorte (clip)

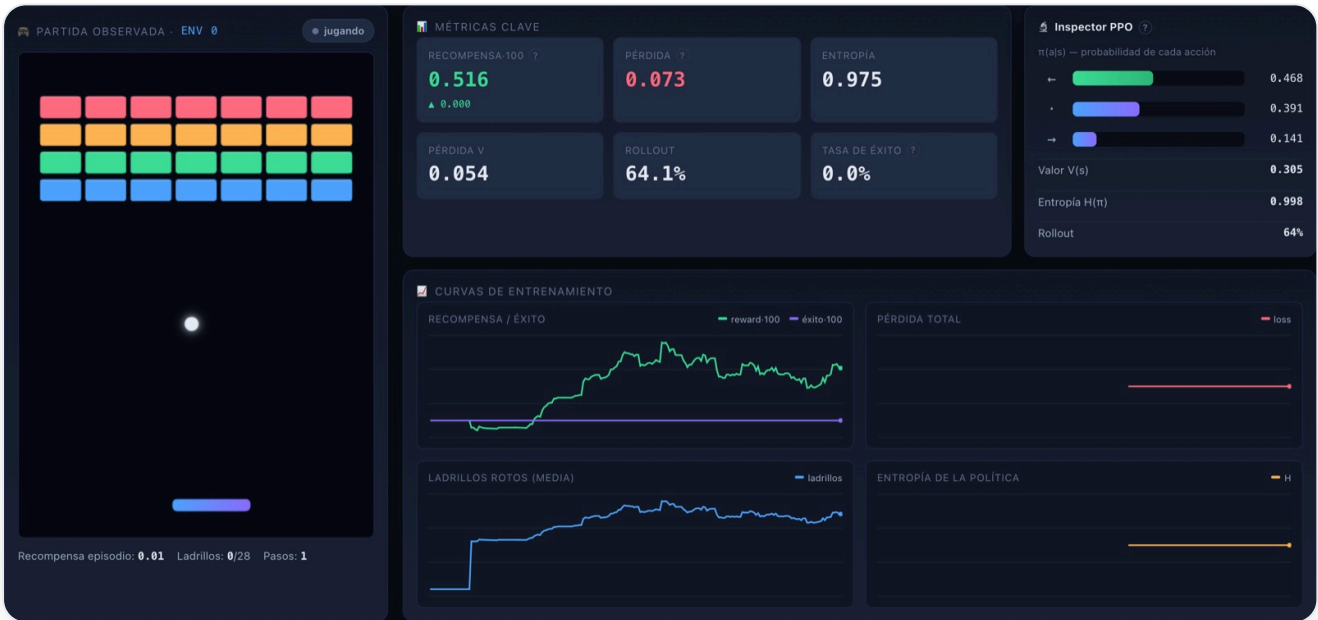
PPO mide cuánto cambia la política (ratio ρ) y **recorta** ese cambio a $[1-\epsilon, 1+\epsilon]$ (con $\epsilon=0.2$, a $[0.8, 1.2]$). Así nunca da un paso tan grande que arruine la política. Las ventajas se estiman con **GAE** (combina errores TD de varios pasos) para reducir la varianza.

El experimento: objetivo, entorno y parámetros

Mismo entorno Arkanoid. PPO es **on-policy**: recoge un rollout, lo exprime en varias épocas y lo tira.

AJUSTE	VALOR	QUÉ ES Y QUÉ EFECTO TIENE
<code>longitud_rollout</code>	256	Pasos por entorno que se recogen antes de cada actualización.
<code>épocas</code>	4	Cuántas veces se reaprovecha cada rollout antes de descartarlo.
<code>clip ϵ</code>	0.2	Cuánto se permite cambiar la política por actualización (el "freno").
<code>GAE λ</code>	0.95	Equilibrio sesgo/varianza al estimar la ventaja (0 = solo 1 paso; $\rightarrow 1$ = muchos pasos).
<code>coef_entropía</code>	0.01	Premia mantener algo de aleatoriedad en la política (exploración).
<code>max_norma_grad</code>	0.5	Recorta el gradiente para que ningún paso sea desmesurado.

La interfaz



PPO entrenando: el inspector muestra $\pi(a|s)$ como barras de probabilidad, el valor $V(s)$ y la entropía. Las curvas se adaptan (entropía en vez de ϵ).

Inspector PPO ?

$\pi(a|s)$ — probabilidad de cada acción

←



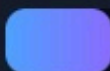
0.468

·



0.391

→



0.141

Valor $V(s)$

0.305

Entropía $H(\pi)$

0.998

Rollout

64%

Inspector de PPO: probabilidades de cada acción, valor estimado del estado y entropía de la política.

Resultados y la mejora con agentes

PPO aprende de forma muy **estable** (curva monótona). En ejecución real, la recompensa pasó de ~ 0.1 a ~ 0.99 en el navegador, y en una corrida más larga en CPU alcanzó ~ 4.08 (ya rompe varios ladrillos por episodio). La entropía se mantiene alta al principio y baja despacio según la política se afila.



Curvas reales de PPO. La gráfica de "datos" muestra los ladrillos rotos de media; la de exploración, la entropía de la política.

⚠ El problema

Optimizar la política sin freno provoca pasos enormes que **destrozan** lo aprendido: la recompensa colapsa de golpe.

✓ La mejora encontrada

El **objetivo recortado + normalizar las ventajas + recorte de gradiente**. Resultado: la curva más estable de los cuatro, sin caídas bruscas.

Conclusiones

En una frase

PPO es **on-policy** y **actor-crítico**: mejora la política a pasitos **recortados**, usa la **ventaja** (GAE) para saber qué acciones reforzar y premia la **entropía** para no dejar de explorar. Estable y de uso general.

SAC — Soft Actor-Critic (discreto)

Maximiza recompensa Y aleatoriedad, y se autorregula.

¿Qué es SAC?

SAC es actor-crítico **off-policy** con una idea extra: no solo maximiza la recompensa, también la **entropía** de la política (cuán aleatoria es). Esto mantiene la exploración viva y evita que el agente se "cierre" demasiado pronto en una sola estrategia. Usa **dos críticos** y un coeficiente de temperatura α que **se ajusta solo**.



Una analogía

Como un chef que ya tiene un plato que funciona pero sigue probando variaciones por si alguna es mejor: SAC se reserva, a propósito, una dosis de "probar cosas nuevas" mientras le compense.

¿Para qué sirve? (en el mundo real)



Robótica continua

Es uno de los mejores algoritmos para control de robots reales por su eficiencia de muestra y robustez.



Conducción y navegación

Entornos donde explorar de forma segura y constante es clave.



Control fino

Tareas con muchas soluciones parecidas, donde mantener variedad ayuda a no estancarse.



Investigación

Banco de pruebas habitual para estudiar exploración y estabilidad.

Cómo funciona (su estructura)

Un actor (política) y **dos críticos** Q (más sus dos redes objetivo). Trabajar con el **mínimo** de los dos críticos evita la sobreestimación de valores.

Actor

$\rightarrow \pi(a|s) \cdot 3$

+

Crítico Q1

$\rightarrow 3$

+

Crítico Q2

$\rightarrow 3$

+

Temperatura α

auto

$y = r + \gamma \cdot \sum_a \pi(a|s') \cdot [\min(Q1', Q2') - \alpha \cdot \log \pi(a|s')]$ // objetivo del crítico

La idea clave: temperatura automática

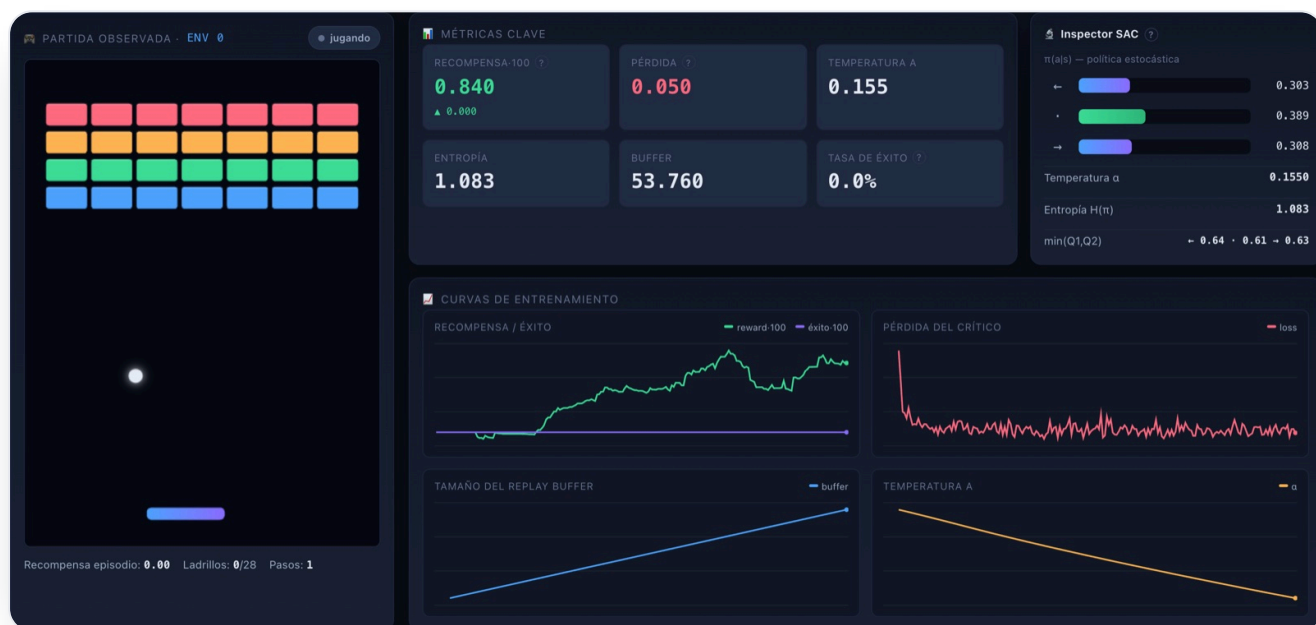
α pondera cuánta entropía exigimos. En vez de fijarlo a mano, SAC lo **aprende** para alcanzar una entropía objetivo (aquí, $0.55 \cdot \log 3$). Si la política explora poco, α sube; si explora demasiado, baja.

En nuestras ejecuciones, α bajó de 0.20 a **~0.165** sola.

El experimento: objetivo, entorno y parámetros

AJUSTE	VALOR	QUÉ ES Y QUÉ EFECTO TIENE
lr_actor / crítico / α	6e-4 / 8e-4 / 6e-4	Tres ritmos de aprendizaje, uno por red. El crítico aprende algo más rápido.
tau (τ)	0.01	Velocidad de las dos redes objetivo (soft update).
entropía objetivo	$0.55 \cdot \log 3$	Cuánta aleatoriedad queremos mantener; gobierna hacia dónde empuja α .
batch / buffer	128 / 100.000	Es off-policy: reutiliza experiencias de un replay buffer.

La interfaz

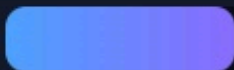


SAC entrenando: el inspector muestra la política, la temperatura α y la entropía. La curva de "exploración" sigue a α en vez de a ϵ .

Inspector SAC ?

$\pi(a|s)$ — política estocástica

←



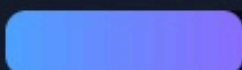
0.303

·



0.389

→



0.308

Temperatura α

0.1550

Entropía $H(\pi)$

1.083

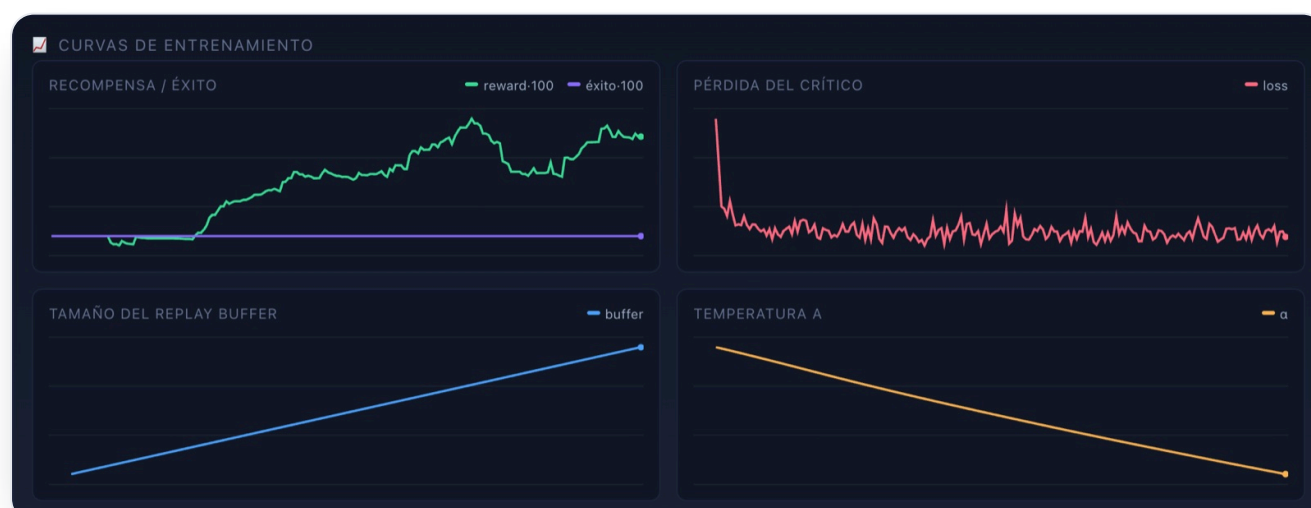
$\min(Q1, Q2)$

← 0.64 · 0.61 → 0.63

Inspector de SAC: $\pi(a|s)$, temperatura α (auto-ajustada), entropía y el $\min(Q1, Q2)$ por acción.

Resultados y la mejora con agentes

SAC aprende y la temperatura se estabiliza sola. En el navegador, la recompensa subió de ~ 0.0 a ~ 0.99 , con α descendiendo de 0.20 a ~ 0.165 y la entropía manteniéndose alta (exploración sana). Sin fugas: 76 tensores constantes durante todo el entrenamiento.



Curvas reales de SAC. La gráfica de exploración muestra la temperatura α descendiendo poco a poco.

⚠ El problema

SAC discreto puede **diverger** (los Q se disparan) o quedarse demasiado aleatorio/demasiado rígido si α está mal puesto a mano.

✓ La mejora encontrada

Doble crítico con mínimo (anti-sobreestimación) + **α automática** (no hay que afinarla). Resultado: aprendizaje estable y exploración auto-regulada.

I Conclusiones

En una frase

SAC es **off-policy** y de **máxima entropía**: usa **dos críticos** para no sobreestimar y una **temperatura α automática** para mantener la exploración justa. Muy eficiente en datos y robusto.

World Model — Dyna-Q

Aprende un modelo del juego y entrena "imaginando".

¿Qué es un World Model?

Los tres anteriores son **model-free**: aprenden qué hacer sin entender el entorno. Un World Model es **model-based**: aprende un **modelo de la dinámica** —una red que predice, dado (s, a) , cuál será el siguiente estado s' , la recompensa r y si termina— y lo usa para generar experiencia **imaginada** con la que entrenar (Dyna-Q). Así exprime cada paso real muchas veces.



Una analogía

Como un ajedrecista que, además de jugar, **imagina jugadas en su cabeza** para practicar sin mover ficha. El World Model "sueña" partidas con su modelo del mundo y aprende también de esos sueños.

¿Para qué sirve? (en el mundo real)



Cuando los datos son caros

Robots reales o procesos donde cada paso cuesta tiempo/dinero: imaginar multiplica los datos disponibles.



Planificación

Modelos del mundo (MuZero, Dreamer) que planifican mirando hacia delante con su propio simulador aprendido.



Eficiencia de muestra

Aprender buenas políticas con muchos menos episodios reales.



Simulación aprendida

Sustituir un simulador caro por una red rápida que lo aproxima.

Cómo funciona (su estructura)

Dos piezas: un **Q-net** (como DQN) que decide, y un **modelo de dinámica** ($6+3$ entradas $\rightarrow 200 \rightarrow 200 \rightarrow \Delta s, r, \text{done}$) que predice el futuro. El modelo predice el **incremento** Δs (más estable que s' directo): $s' = s + \Delta s$.



real $\rightarrow$ entrena modelo y Q ; imaginado $\rightarrow$ 5 actualizaciones Q extra por paso real (planning)

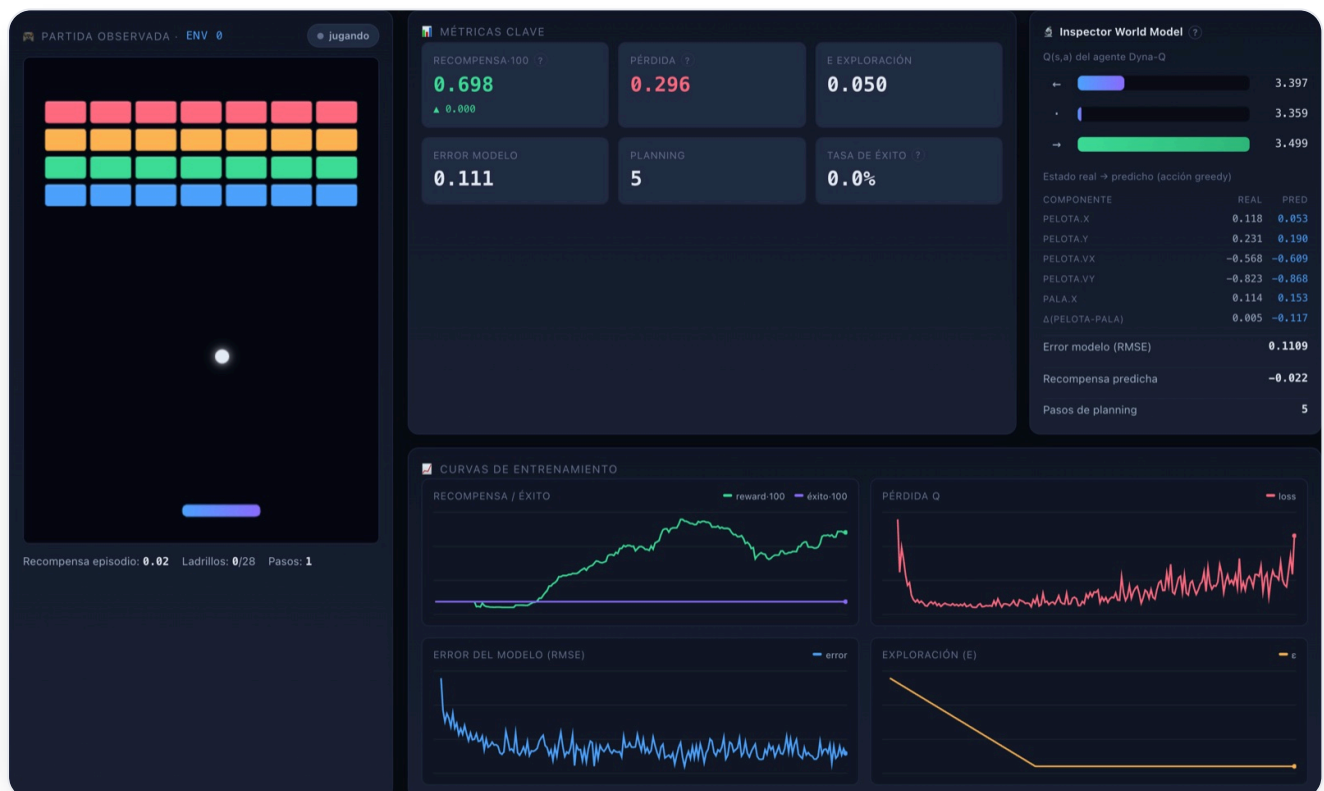
La idea clave: planning con datos imaginados

Por cada paso real hacemos **5** actualizaciones del Q-net con transiciones que **genera el modelo**, partiendo de estados muestreados del buffer. Si el modelo es bueno, es casi gratis multiplicar el aprendizaje.

El experimento: objetivo, entorno y parámetros

AJUSTE	VALOR	QUÉ ES Y QUÉ EFECTO TIENE
capas del modelo	200-200	El modelo de dinámica es algo más grande: tiene que aprender la física.
pasos_planning	5	Actualizaciones del Q-net con datos imaginados por cada paso real.
arranque_modelo	1.000	Pasos reales antes de empezar a entrenar el modelo (necesita datos primero).
lr_modelo	1e-3	Ritmo de aprendizaje del modelo de dinámica (supervisado).

La interfaz



World Model entrenando: las métricas se adaptan (aparecen "Error modelo" y "Planning"). El inspector compara el estado real con el predicho por el modelo.

Inspector World Model ?

Q(s,a) del agente Dyna-Q

←		3.397
·		3.359
→		3.499

Estado real → predicho (acción greedy)

COMPONENTE	REAL	PRED
PELOTA.X	0.118	0.053
PELOTA.Y	0.231	0.190
PELOTA.VX	-0.568	-0.609
PELOTA.VY	-0.823	-0.868
PALA.X	0.114	0.153
$\Delta(\text{PELOTA-PALA})$	0.005	-0.117

Error modelo (RMSE) 0.1109

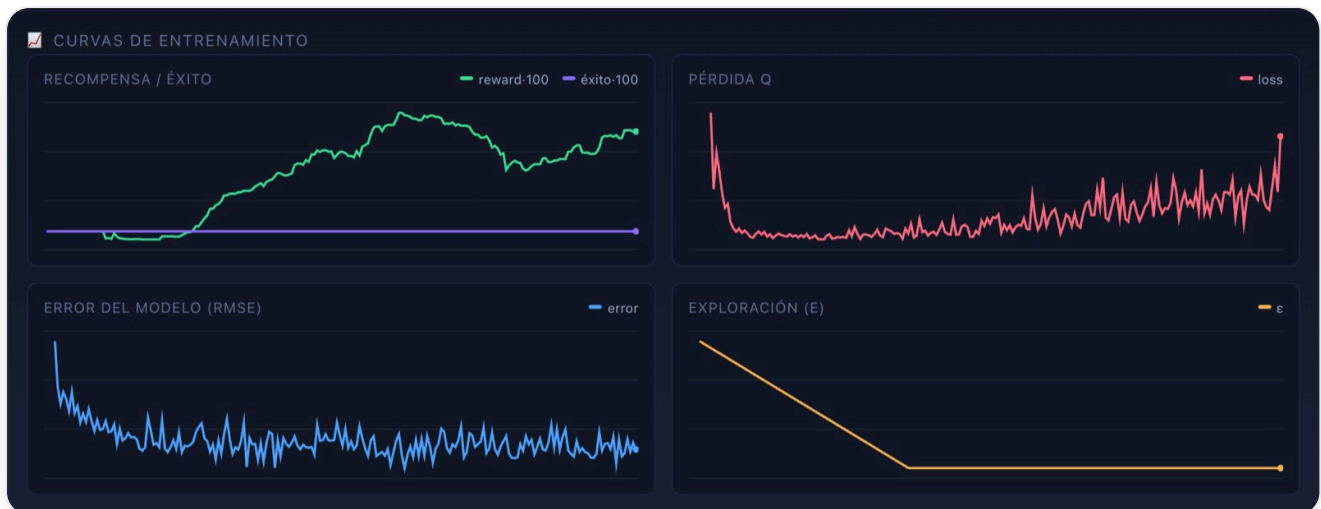
Recompensa predicha -0.022

Pasos de planning 5

Inspector del World Model: Q-values del Dyna-Q y la tabla real → predicho componente a componente. Aquí el error RMSE del modelo es 0.11.

Resultados y la mejora con agentes

El modelo aprende a predecir la física con un error **RMSE ~ 0.066** sobre estados normalizados (muy bajo: las predicciones siguen de cerca a la realidad, como se ve en el inspector). La recompensa subió de ~ 0.0 a **~ 0.71** en el navegador. Sin fugas: ~ 46 tensores constantes.



Curvas reales del World Model. La gráfica de "datos" muestra el error del modelo (RMSE) bajando: el modelo mejora su predicción del juego.

⚠ El problema

El **model bias**: si el modelo se equivoca, las imaginaciones largas acumulan error y el agente aprende de un mundo falso.

✓ La mejora encontrada

Predecir el **incremento Δs** (más estable) e imaginar a **corto plazo** (1 paso, reiniciando trayectorias terminadas). Resultado: modelo fiable (RMSE bajo) y política que mejora con datos imaginados.

Conclusiones

En una frase

El World Model es **model-based**: aprende un **modelo de dinámica** y lo usa para **imaginar** experiencia (Dyna-Q), ganando eficiencia de muestra. El precio a vigilar es el **model bias**.

COMPARADOR

Los cuatro, cara a cara

Misma tarea, mismo entorno, cuatro filosofías distintas.

Algoritmo	Familia	Política	Qué aprende	Exploración	Señal propia	Reward real*
DQN	Model-free · valor	off-policy	$Q(s,a)$	ϵ -greedy	TD-error, ϵ	$0 \rightarrow 0.94$
PPO	Model-free · actor-crítico	on-policy	$\pi(a s) + V(s)$	entropía	entropía, ventaja	$0.1 \rightarrow 0.99$ (4.08 en CPU)
SAC	Model-free · actor-crítico	off-policy	$\pi + 2 \cdot Q$	máx. entropía (α)	temperatura α	$0 \rightarrow 0.99$
World Model	Model-based	off-policy	dinámica + Q	ϵ -greedy	error del modelo	$0 \rightarrow 0.71$

* Recompensa media de los últimos 100 episodios, medida en ejecuciones reales (navegador con WebGPU; el dato de PPO en CPU es de una corrida más larga). El throughput observado fue de ~24.000 exp/s en WebGPU frente a ~5.600 exp/s en CPU.

Qué llevarse

No hay un "mejor" universal: **DQN** es el cimiento más simple; **PPO** es el más estable y de uso general; **SAC** es el más eficiente en datos y explora solo; el **World Model** es el más ambicioso (aprende el juego) y el más delicado. Entender los cuatro es entender el mapa del RL profundo.

PRUÉBALO

Descárgalo y entrénalo tú

Todo corre en local; el entrenamiento, en tu navegador con WebGPU.

Repositorio

github.com/robertocanalesmora/arkanoid-drl-learning-lab

1 · Clona e instala

terminal

INSTALACIÓN

```
git clone «URL-del-repo» && cd DRLArkanoid  
npm install           # vite + @tensorflow/tfjs + backend WebGPU
```

2 · Arranca el laboratorio

terminal

ARRANCAR

```
npm run dev           # abre http://localhost:5173 y pulsa ► Entrenar  
./arrancar.sh         # alternativa: arranca y, si ya estaba, rearranca
```

3 · Verifica o entrena por consola (sin navegador)

terminal

VERIFICACIÓN

```
npm run verificar     # entrena los 4 algoritmos (CPU) y reporta si aprenden  
node scripts/entrenar.mjs dqn --pasos 40000 --envs 128 # entrena uno y muestra trazas
```

Requiere Node ≥ 18 y un navegador con WebGPU (Chrome/Edge/Safari recientes). Si no hay WebGPU, el laboratorio cae automáticamente a WebGL y, en último caso, a CPU.

Cómo está organizado el código

Dónde vive cada pieza del laboratorio.

```
src/
├── nucleo/                # infraestructura común
│   ├── constantes.js      # entorno, recompensas, hiperparámetros
│   ├── busEventos.js      # pub/sub: desacopla motor y UI
│   ├── gestorTensores.js  # tf.tidy, soft update, paso de gradiente
│   └── trazas.js          # logging estructurado de métricas
├── entorno/
│   ├── entornoArkanoid.js # el juego (física de paso fijo)
│   └── gestorEntornos.js  # pools headless + visual
├── agentes/              # ←* AQUÍ VIVEN LAS REDES Y LOS ALGORITMOS
│   ├── agenteDQN.js       # Double DQN
│   ├── agentePPO.js       # actor-crítico + GAE + clip
│   ├── agenteSAC.js       # 2 críticos +  $\alpha$  automática
│   ├── agenteWorldModel.js # modelo de dinámica + Dyna-Q
│   └── redes/constructorRedes.js # fábrica de MLPs
├── datos/
│   ├── replayBuffer.js    # off-policy (DQN, SAC, WM)
│   ├── replayPrioritario.js # PER con sum-tree
│   └── rolloutBuffer.js   # on-policy (PPO) + GAE
├── entrenamiento/        # ←* EL BUCLE DE ENTRENAMIENTO
│   ├── orquestador.js     # el ciclo, sin DOM
│   └── metricas.js        # recolector de métricas reales
├── ui/                   # interfaz (paneles, curvas, inspectores)
└── main.js               # arranque + detección de backend
```

Las redes agentes/*.js + agentes/redes/constructorRedes.js

El entrenamiento entrenamiento/orquestador.js (el ciclo) y el método entrenar() de cada agente (la regla de actualización).

El juego entorno/entornoArkanoid.js (física) y gestorEntornos.js (los dos pools).

La memoria datos/replayBuffer.js (off-policy) y datos/rolloutBuffer.js (on-policy).

Algoritmo a algoritmo

Fragmentos reales: la red, el entrenamiento y un extra de cada uno.

La fábrica de redes (común a todos)

src/agentes/redes/constructorRedes.js · crearMLP()

LA RED

```
const modelo = tf.sequential({ name: nombre });
capasOcultas.forEach((unidades, i) => {
  modelo.add(tf.layers.dense({
    units: unidades, activation: "relu",
    inputShape: i === 0 ? [dimEntrada] : undefined,
    kernelInitializer: "heNormal", // buena inicialización para ReLU
  }));
});
modelo.add(tf.layers.dense({ units: dimSalida, activation: activacionSalida }));
```

Un MLP parametrizable. DQN/WM lo usan con 3 salidas (Q); PPO/SAC para el actor (3 logits) y la crítica (1 valor).

PPO · el objetivo recortado

src/agentes/agentePPO.js · entrenar() (minibatch)

EL ENTRENAMIENTO

```
const ratio = logpA.sub(logpViejoMb).exp(); //  $\rho = \pi_{nueva} / \pi_{vieja}$ 
const surr1 = ratio.mul(advMb);
const surr2 = ratio.clipByValue(1 - eps, 1 + eps).mul(advMb); // recorte
const lossPol = surr1.minimum(surr2).mean().mul(-1); // objetivo recortado
const entropia = probs.mul(logp).sum(1).mean().mul(-1);
const lossVal = valor.sub(retMb).square().mean(); // crítica: MSE contra el retorno
return lossPol.add(lossVal.mul(coefValor)).sub(entropia.mul(coefEntropia));
```

El corazón de PPO: tomar el mínimo entre el objetivo normal y el recortado evita pasos demasiado grandes.

SAC · la temperatura α que se aprende sola

src/agentes/agenteSAC.js · entrenar() (paso de α)

UN EXTRA

```
//  $\alpha$  se ajusta para acercar la entropía de la política a la entropía objetivo.
const lossAlphaT = pasoGradiente(this.optAlpha, () => {
  const term = this.logAlpha.mul(-1).mul(logpC.add(this.targetEntropy));
  return probsC.mul(term).sum(1).mean();
}, [this.logAlpha]);
```

α no es un hiperparámetro fijo: es una variable entrenable. Esto le quita al usuario el trabajo de afinarla.

World Model · entrenar el modelo de dinámica

src/agentes/agenteWorldModel.js · _entrenarModelo()

UN EXTRA

```
const lossT = pasoGradiente(this.optModelo, () => {
  const out = this.modelo.predict(entrada);           // entrada = [s @ one-hot(a)]
  const dsPred = out.slice([0, 0], [B, this.dimEstado]); //  $\Delta s$  predicho
  const rPred = out.slice([0, this.dimEstado], [B, 1]); // r predicha
  const donePred = out.slice([0, this.dimEstado + 1], [B, 1]);
  return tf.losses.meanSquaredError(dsObjetivo, dsPred) // aprender la física...
    .add(tf.losses.meanSquaredError(rT, rPred))         // ...la recompensa...
    .add(tf.losses.sigmoidCrossEntropy(doneT, donePred).mul(0.5)); // ...y el fin de episodio
}, this._varsModelo);
```

El modelo se entrena de forma **supervisada**: predecir (Δs , r, done) a partir de (s, a). Con él, el agente "imagina".

Licencia MIT

Úsalo, modifícalo y compártelo libremente.

Autor	Roberto Canales Mora · con Claude Chat / Code
Licencia	MIT
Repositorio	github.com/robertocanalesmora/arkanoid-drl-learning-lab
Entorno	Vite + TensorFlow.js (WebGPU / WebGL / CPU). Probado en macOS (Apple Silicon M4).

LICENSE

TEXTO COMPLETO

Licencia MIT

Copyright (c) 2026 Roberto Canales Mora

Por la presente se concede permiso, sin cargo, a cualquier persona que obtenga una copia de este software y de los archivos de documentación asociados (el "Software"), para utilizar el Software sin restricción, incluyendo sin limitación los derechos a usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del Software, y a permitir a las personas a las que se les proporcione el Software a hacer lo mismo, sujeto a las siguientes condiciones:

El aviso de copyright anterior y este aviso de permiso se incluirán en todas las copias o partes sustanciales del Software.

EL SOFTWARE SE PROPORCIONA "TAL CUAL", SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA, INCLUYENDO PERO NO LIMITADO A GARANTÍAS DE COMERCIALIZACIÓN, IDONEIDAD PARA UN PROPÓSITO PARTICULAR Y NO INFRACCIÓN. EN NINGÚN CASO LOS AUTORES O PROPIETARIOS DE LOS DERECHOS DE AUTOR SERÁN RESPONSABLES DE NINGUNA RECLAMACIÓN, DAÑOS U OTRAS RESPONSABILIDADES, YA SEA EN UNA ACCIÓN DE CONTRATO, AGRAVIO O DE OTRO MODO, QUE SURJA DE, FUERA DE O EN CONEXIÓN CON EL SOFTWARE O EL USO U OTROS NEGOCIOS EN EL SOFTWARE.